

From Vibe Coding to Spec-Driven Agentic Engineering

A Hands-On Implementation using Claude Code

PART II

Instructor: Muhammad Arif Butt, PhD

About The Instructor



Dr. Muhammad Arif Butt
Asst. Prof. Department of Data Science
University of Punjab, Lahore
arif@pucit.edu.pk
https://arif.excaliat@gmail.com
https://arifbutt.me
https://www.linkedin.com/in/dr-arif-butt/
https://youtube.com/learnwitharif
https://github.com/arifpucit
https://huggingface.co/arif-butt

- **Education:**
  - Graduation from Pakistan Military Academy, Kakul
  - MPhil CS from University of Punjab, Lahore
  - PhD CS from University of Punjab, Lahore
- **Experience:**
  - Served in field/staff/instructional posts in Pakistan Army
  - Assistant Professor, Department of Data Science
  - Founder Excaliat (https://excaliat.com/en)
  - Founder FalconHunt (https://falconhunt.org/)
  - Co-Founder Tbox Solutionz (https://tboxsolutionz.com/)
- **Research Interest:**
  - Embedded and Real-Time Operating Systems
  - Vulnerability Analysis, Binary Exploitation & Exploit Development
  - AI-Driven Cybersecurity and Securing AI systems

Scheme of Sessions

Part- I: Overview of Agentic Coding Assistants

Part– II: Claude Ecosystem, Installing & using Claude Code

Part– III: Context Window, Slash Commands & Claude Directory

Part– IV: A Vulnerable Web Application & its Attack Vectors

Part– V: Workflow of Implementing Features & Fixing Bugs

Part– VI: Skill Market place and Writing your Own Skills

Part– VII: Creating and using Subagents and Agent Teams

Part– VIII: Model Context Protocol

Part– IX: Creating and using Hooks

Part– X: Creating and using Plugins

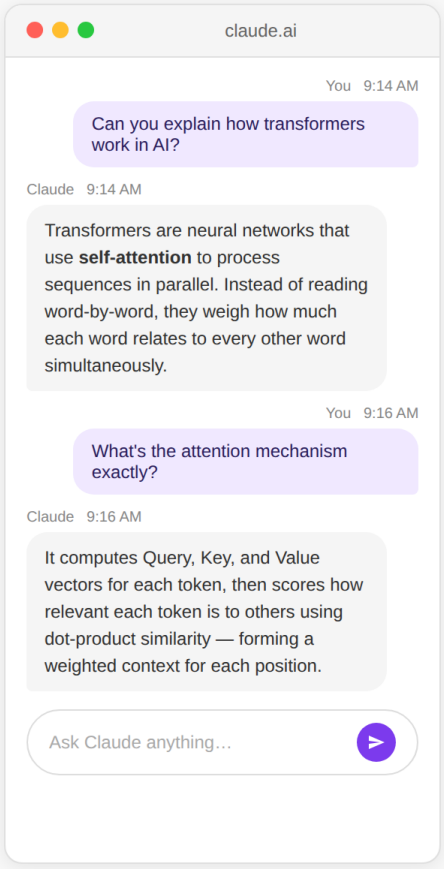
Part 2
Claude Ecosystem
Installing & using Claude Code

Instructor: Muhammad Arif Butt, PhD

Claude Ecosystem

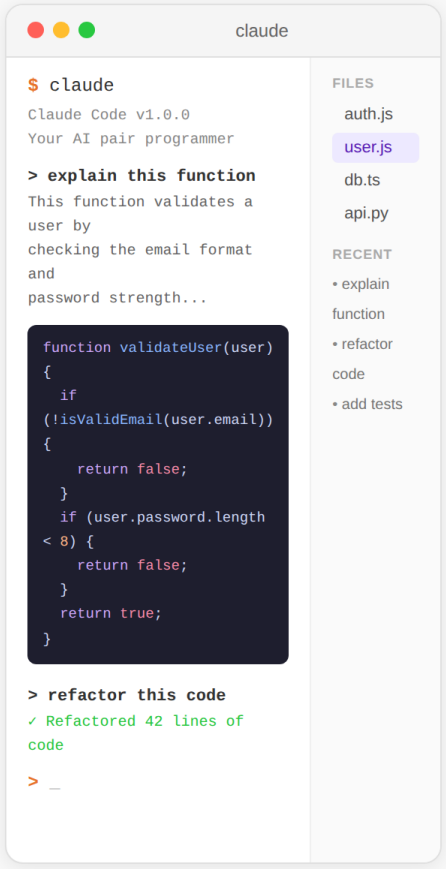
Claude Chat

Ask anything, write anything, understand anything.



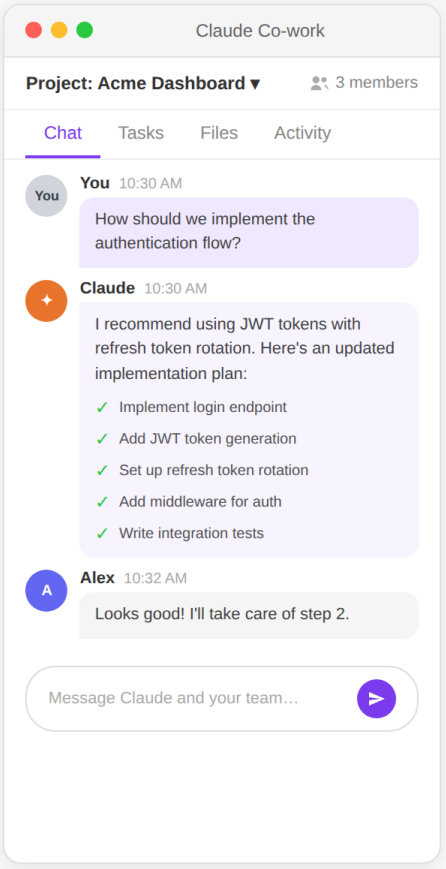
Claude Code

AI-powered coding assistant in your terminal.



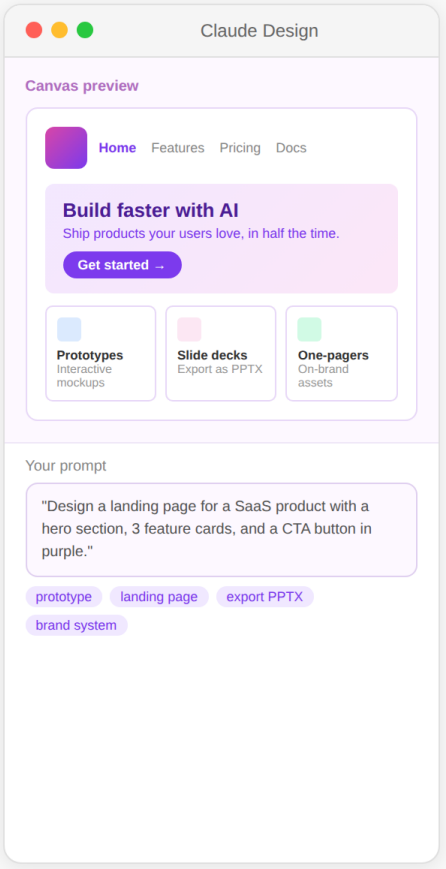
Claude Co-work

Collaborate with Claude and your team.



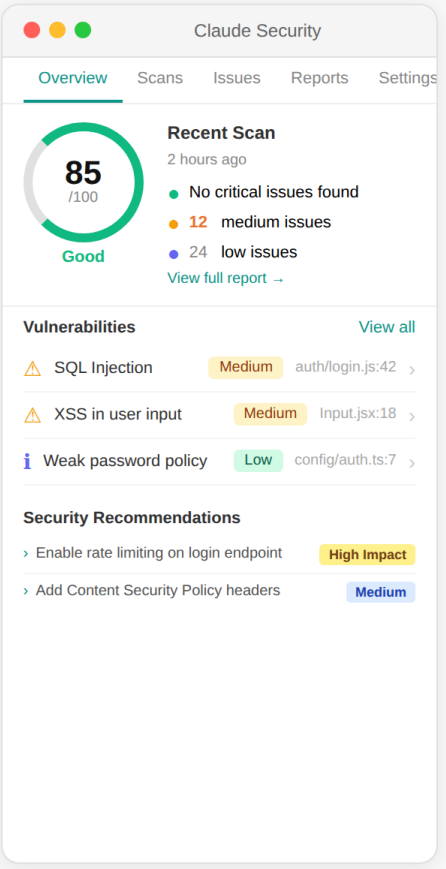
Claude Design

Create visuals, prototypes, and decks with AI.



Claude Security

AI-powered security analysis and threat detection.



How to Access each Product?

Claude Chat

SUBSCRIPTION PLANS

- Free tier (limited daily usage)
- Pro — $20/month
- Max 5x — $100/month
- Max 20x — $200/month
- Team — $25/seat/month
- Enterprise (custom pricing)

API ACCESS

- **YES — Per-token billing**

Models: claude-opus-4-7, claude-sonnet-4-6, claude-haiku-4-5
Also: AWS Bedrock, GCP Vertex AI

OTHER ACCESS

- claude.ai (web)
- Desktop app (Mac/Win)
- Mobile (iOS / Android)
- Claude in Slack

Claude Code

SUBSCRIPTION PLANS

× Not in Free tier

- Pro — $20/month
- Max 5x — $100/month
- Max 20x — $200/month
- Team — $25/seat/month
- Enterprise (custom pricing)

API ACCESS

- **YES — Per-token billing**

Bring-your-own API key. Token usage billed at standard Anthropic rates (no fixed subscription required if using API).

OTHER ACCESS

- Terminal / CLI (native)
- VS Code extension
- JetBrains IDEs
- Claude Code on the Web
- Slack & GitHub Actions

Claude Co-work

SUBSCRIPTION PLANS

× Not in Free tier

- Pro — $20/month
- Max 5x — $100/month
- Max 20x — $200/month
- Team
- Enterprise

API ACCESS

× NO — Not API-billable

Cowork is a packaged product experience, not an API endpoint. Underlying agentic capability lives in the Claude Agent SDK.

OTHER ACCESS

- Desktop app (Mac/Win)
- Dispatch (mobile remote task assignment)

× Not on web

× Not on mobile directly

Claude Design

SUBSCRIPTION PLANS

× Not in Free tier

- Pro — $20/month
- Max 5x — $100/month
- Max 20x — $200/month
- Team
- Enterprise

(Research preview)

API ACCESS

× NO — Not API-billable

Design is a workflow inside claude.ai. The underlying vision model (Opus 4.7) is on the API, but the Design canvas itself is not.

OTHER ACCESS

- claude.ai (web only)
- Design canvas / tab
- Exports to PPTX / PDF

No CLI, no IDE, no API

Claude Security

SUBSCRIPTION PLANS

× Not in Free tier

× Not in Pro

- Max — coming soon
- Team — coming soon
- Enterprise (public beta)

Currently Enterprise-first; broader rollout in progress

API ACCESS

× NO — Not API-billable

No API integration or custom agent build needed — accessed directly from claude.ai/security or the Claude sidebar.

OTHER ACCESS

- claude.ai/security (web)
- Claude sidebar tab
- Admin console for rollout
- Export to PM tools (Jira, Linear, etc.)

https://support.claude.com/en/

How Claude Code Work?

Claude Code (If you can describe it, you can build it)

Your AI coding partner in the terminal

Claude Code understands your codebase, plans changes, uses tools, and delivers results.

1. You Provide a Request

```
// Example request
Add error handling to
this function and write
tests for it.
```

- Your codebase
- Your terminal
- Your goals

2. Claude Code (Models)

Chooses the right model for the task

- **Opus** — Best for complex reasoning & large code changes
- **Sonnet** — Best balance of intelligence & speed
- **Haiku** — Fastest for simple tasks & edits

Claude Code

Uses tools as needed

3. Claude Code Generates a Response

```
try:
  result = risky_operation()
  return result
except Exception as e:
  logger.error(str(e))
  raise
```

Response may include:

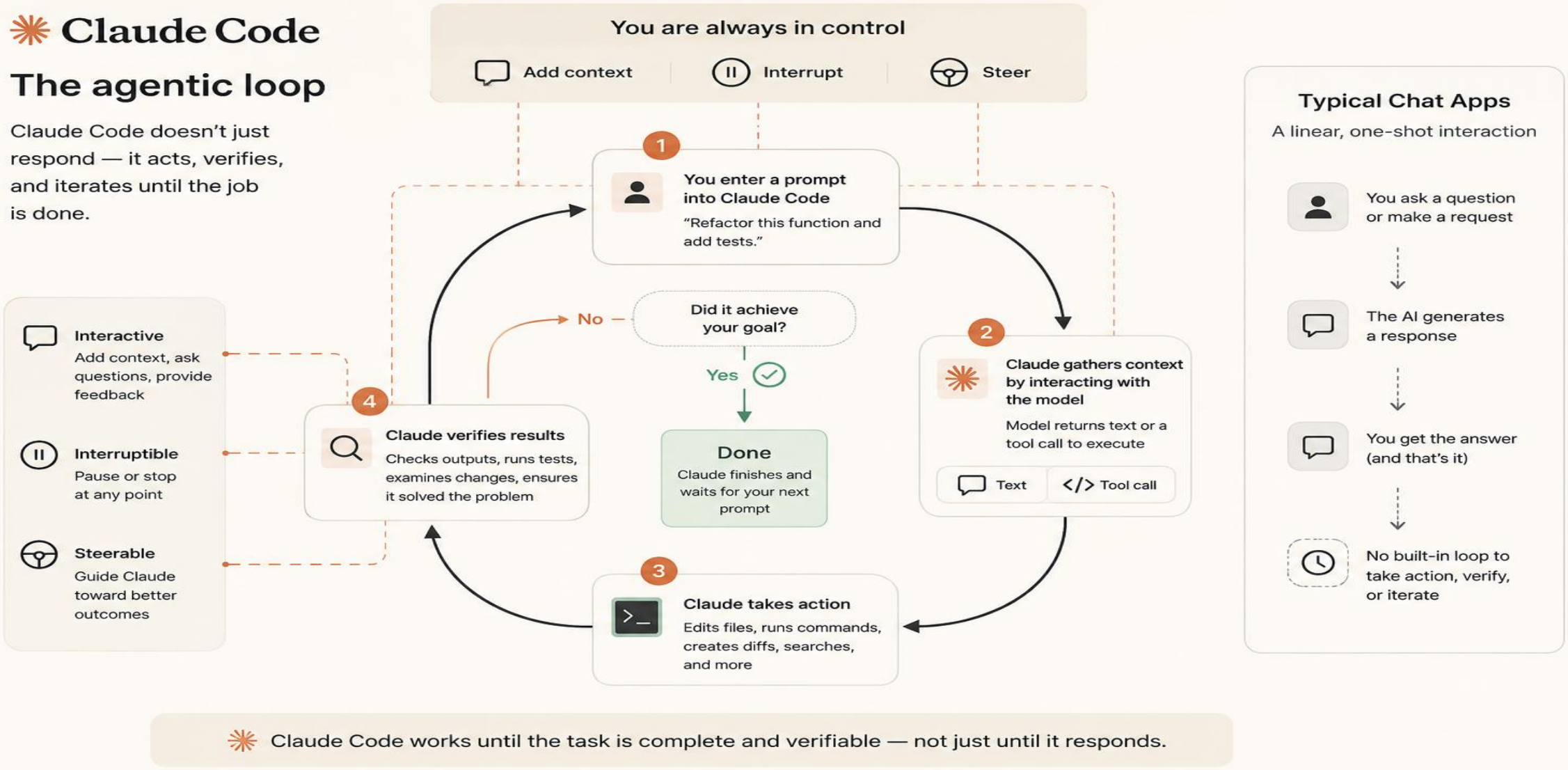
- ✓ Code changes
- ✓ Explanations
- ✓ Tests
- ✓ File edits
- ✓ Commands run
- ✓ Next steps

4. Tools Claude Code Can Use

- **Access local file system** — Read, write, and edit files
- **Web search** — Find up-to-date information
- **Execute bash commands** — Run commands and get results
- **Switch between modes** — Chat / Thinking / Plan
- **Compress past conversation** — Summarize to save context
- **Ask user questions** — Gather more context when needed
- **Commits to repository** — Create commits with changes
- **Open Pull Requests** — Create PRs and link changes

Always respects your permissions, .gitignore, and repository rules.

The Agentic Loop



Installations

Claude Code: https://claude.com/claude-code

MacOS/Linux: `$ curl -fsSL https://claude.ai/install.sh | bash`

Windows PS: `> irm https://claude.ai/install.ps1 | iex`

```
$ which claude
~/.local/bin/claude
```

On MacOS, you may need to add the path of Claude code binary's folder to your path environment variable or inside `~/.zshrc` and then `source ~/.zshrc`. On Windows "Edit environment variables" settings, then open a new terminal window for it to take effect.

Python: https://www.python.org/downloads/

C Compiler: https://sourceforge.net/projects/gcc-win64/

git: https://git-scm.com/install/mac

VS Code: https://code.visualstudio.com/

pip/uv: https://pypi.org/project/pip/, https://pypi.org/project/uv/

Node.js: https://nodejs.org/en/download

```
$ claude --version
$ python -version
$ uv --version
$ gcc --version
$ git --version
$ node --version
$ npm --version
```

Three Ways to Access Claude Code

Choose the login method that fits your setup

ACCOUNT LOGIN
01

Claude Subscription

- Log in with your Claude.ai account
- Individual Pro or Max plan
- Usage included in subscription

`https://claude.ai`

API KEY
02

Anthropic Console

- Create an API key in the Console
- Save in env variable or ~/.zshrc
- Pay-as-you-go billing

`https://console.anthropic.com/settings/keys`

CLOUD PROVIDER
03

Third-Party Platform

- Amazon Bedrock
- Google Vertex AI
- Microsoft Foundry

`https://docs.claude.com/en/docs/claude-code/third-party-integrations`

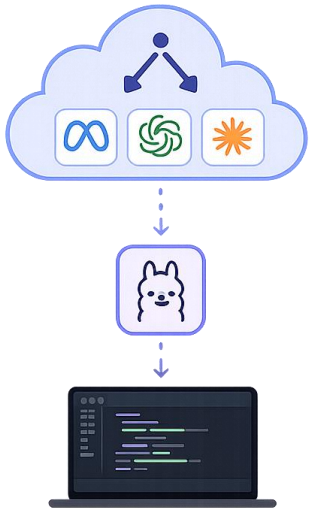
Running Claude with Pro/Max Subscription



Run Claude Code Using Ollama

- **Ollama** is an open-source platform that enables users to download open-source models from the Ollama Hub or Hugging Face Hub, run them locally, and interact with them through a CLI or ollama desktop app or via API.
- Since v0.14 (January 2026), it also supports agentic coding tools like Claude Code run on Ollama-served local or cloud-hosted models through the **`ollama launch`** command.

Solution

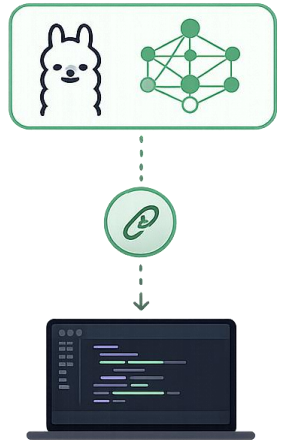


Ollama cloud hosted models
via open router



Ollama local models
via http://localhost:11434





Note: This runs the coding agent on open models like GLM or Qwen, not Anthropic's actual Claude models.

Install Ollama

Ollama: https://ollama.com

MacOS/Linux: `$ curl -fsSL https://ollama.com/install.sh | sh`

Windows PS: `> irm https://ollama.com/install.ps1 | iex`

```
$ which ollama
/usr/local/bin/ollama
```

For macOS and Windows, the installer also includes a GUI app

Capabilities

- Download and pull models
- Chat with local/cloud models
- Drag and drop files and ask questions
- Image input if the chosen Ollama model supports multimodality
- Context length adjustment

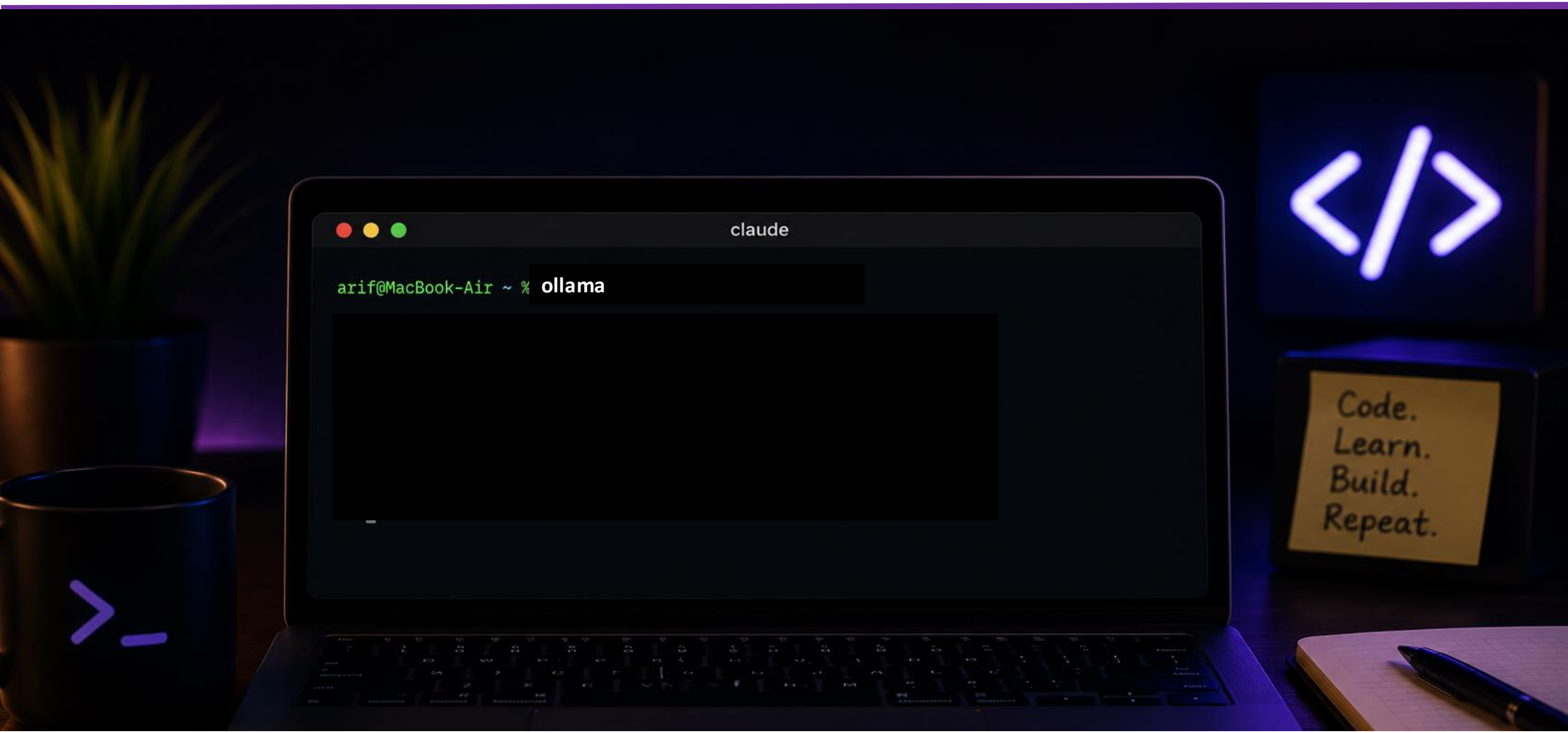


Commands to use Ollama on CLI

- To run ollama use the command: `$ ollama serve`
- Then visit: **http://127:0.0.1:11434** which will display a message “Ollama is running” ☺

```
$ ollama                          # List the possible commands
$ ollama pull llama3.2:1b         # Download the model from https://ollama.com/library and stores it locally
$ ollama list                     # List all the models you have downloaded on your machine
$ ollama show llama3.2:1b         # Display model arch, parameters, context window, embedding length, quantization, capabilities and  license
$ ollama run  llama3.2:1b         # Start an interactive session with the locally hosted model (No internet required)
$ ollama run  gpt-oss:20b-cloud   # Start an interactive session with the cloud hosted model (Internet required)
$ ollama run llama3.2 "prompt" < file.txt   # Ask questions from contents of a file passed via I/O redirection
$ cat file.txt | ollama run llama3.2:1b "prompt"  # Pass contents of a file using pipe operator and ask questions from the passed text
$ curl https://www.arifbutt.me/ | ollama run llama3.2:1b "question"  # Scrape a web page and send the html contents and ask Q
```

Accessing Models using Ollama CLI



Run Claude Code With Ollama

```
$ ollama launch <agent-software> --model <model>
```

Agent Software

- Claude
- Openclaw
- Codex
- Opencode
- Hermes
- PI

Open-Source Models

- Qwen3-coder-next
- GLM-4.7:cloud
- Llama 3
- Gpt-oss:120b-cloud
- Codestral Mamba
- Gemma3:27b-cloud

Closed-Source Models

- GPT-4.1
- Claude Sonnet 4.6
- Gemini 2.5 Pro
- Grok 3
- Command R
- Amazon Nova Pro

Run Claude Code with Ollama



Q/A



Thank you!